

DOM — Attributes, Properties & DOM Node Operations

Attributes vs Properties

Category C — Core Concepts

Attribute vs Property — The Fundamental Difference

Attribute → lives in the HTML markup. It is the *initial value* written in your HTML file.

Property → lives on the DOM JavaScript object. It reflects the *live, current state* of the element in memory.

They start equal but **diverge** the moment the user interacts with the page.

 Think of the attribute as a snapshot from when the page loaded. The property is the live feed of what's happening right now.

```
// HTML: <input id="name" value="Alice" class="form-input">
```

```
const input = document.getElementById("name");
```

```
// --- ATTRIBUTE (from HTML) ---
```

```
input.getAttribute("value"); // "Alice" — initial HTML value
```

```
input.getAttribute("class"); // "form-input" — raw string
```

```
// --- PROPERTY (live DOM state) ---
```

```
input.value; // "Alice" — same at first
```

```
input.className; // "form-input" — same at first
```

```
// User types "Bob" in the input field. Now:
```

```
input.getAttribute("value"); // still "Alice" (HTML didn't change)
```

```
input.value; // "Bob" (live DOM property updated!)
```

⚠️ `getAttribute("value")` = **initial state**. `input.value` = **current state**. Never confuse them.

🔧 `setAttribute` / `removeAttribute` / `hasAttribute`

These three methods let you **read, write, and remove attributes** directly on the HTML element. They operate on the attribute (the HTML string), not the DOM property.

Method	What it does
<code>setAttribute(name, value)</code>	Sets or updates an attribute
<code>removeAttribute(name)</code>	Removes the attribute entirely
<code>hasAttribute(name)</code>	Returns true or false

```
const btn = document.querySelector("button");
```

```
// setAttribute — sets/updates an attribute
```

```
btn.setAttribute("disabled", ""); // disables the button
```

```
btn.setAttribute("aria-label", "Submit"); // sets accessibility label
```

```
btn.setAttribute("data-id", "42"); // sets a custom data attribute
```

```
// removeAttribute — removes the attribute entirely
```

```
btn.removeAttribute("disabled"); // re-enables the button
```

```
// hasAttribute — returns true/false
```

```
btn.hasAttribute("disabled"); // false (we just removed it)
```

```
btn.hasAttribute("aria-label"); // true
```

```
// Practical: toggle disabled based on a condition
```

```
if (inputIsEmpty) {
```

```
  btn.setAttribute("disabled", "");
```

```
} else {
```

```
  btn.removeAttribute("disabled");
```

```
}
```

💡 For boolean attributes like `disabled`, `checked`, `readonly` — their presence means true, absence means false. The value doesn't matter; just use `""` as the value when setting.

data-* Attributes → dataset API

Any attribute starting with data- is a **custom data attribute**. It lets you embed extra metadata directly in HTML elements without misusing standard attributes.

All data-* attributes on an element are accessible via element.dataset — a DOMStringMap object.

Key rule: Kebab-case in HTML automatically becomes camelCase in JavaScript.

```
data-user-id → dataset.userId
data-is-admin → dataset.isAdmin
data-bg-color → dataset.backgroundColor
```

```
// HTML: <div id="card" data-user-id="7" data-role="admin" data-
active="true">
```

```
const card = document.getElementById("card");
```

```
// Reading via dataset (auto camelCase conversion)
card.dataset.userId; // "7" (data-user-id → dataset.userId)
card.dataset.role; // "admin"
card.dataset.active; // "true" (always a string!)
```

```
// Writing via dataset
card.dataset.userId = "99"; // updates data-user-id="99" in HTML
card.dataset.theme = "dark"; // adds data-theme="dark" to HTML
```

```
// Deleting a data attribute
delete card.dataset.active; // removes data-active from HTML
```

```
// Also readable with getAttribute (raw attribute string)
card.getAttribute("data-user-id"); // "99" — same result, different API
```

 dataset values are **always strings**. Parse with Number() or JSON.parse() when you need a real type.

 **getAttribute("class") vs element.className — NOT the same**

Method	Returns	When to use
<code>getAttribute("class")</code>	Raw HTML string, may be null	Reading the original HTML attribute
<code>element.className</code>	Live string, never null	Replacing all classes at once
<code>element.classList</code>	DOMTokenList (array-like)	Adding / removing / toggling individual classes 

```
const el = document.querySelector(".card");
```

```
// classList — the preferred modern API
el.classList.add("active");           // adds a class
el.classList.remove("hidden");       // removes a class
el.classList.toggle("dark");         // adds if absent, removes if present
el.classList.contains("active");     // true / false
el.classList.replace("old", "new");  // swap one class for another
```

```
// className — replaces ALL classes at once (destructive)
el.className = "card active";        // replaces everything
```

```
// getAttribute — raw string from HTML
el.getAttribute("class");            // "card active"
```

 **Always use classList** for individual class manipulation. `className` wipes all existing classes — use it only when you want to set everything from scratch.

Live Demo — Theme Toggle (`classList` + `dataset`)

```
function toggleTheme() {
  const box = document.getElementById("themeBox");
  const isDark = box.dataset.theme === "dark";

  // Update the data-* attribute
  box.dataset.theme = isDark ? "light" : "dark";

  // Toggle a CSS class
  box.classList.toggle("dark-mode", !isDark);
}

// Reading both ways — they return the same value
```

```
box.dataset.theme;           // "dark"
box.getAttribute("data-theme"); // "dark"
```

Creating, Inserting & Removing Nodes

"React yahi karta hai under the hood — har render pe"

Category A — Creating Nodes

document.createElement(tag)

Definition: Creates a brand-new HTML element **in memory**. It does not appear on the page yet — you must insert it somewhere after creating it.

```
const div = document.createElement("div");
const btn = document.createElement("button");
const img = document.createElement("img");
const para = document.createElement("p");

// Set properties before inserting
btn.textContent = "Click me";
btn.setAttribute("type", "button");
img.src = "photo.png";
img.alt = "A photo";

// Element still not visible — needs to be inserted into the DOM
document.body.appendChild(btn); // now it appears on the page
```

 Always configure your element (set text, classes, attributes) **before** inserting it. This avoids multiple reflows.

document.createTextNode(text)

Definition: Creates a **raw text node** — pure text with no wrapping HTML tag. Useful when you need to add text to an element programmatically without using `innerHTML` (which is a security risk with user-generated input).

```
const text = document.createTextNode("Hello, World!");

// Typically appended inside an element
```

```
const p = document.createElement("p");
p.appendChild(text); // <p>Hello, World!</p>
```

// vs innerHTML (avoid with user input — XSS risk!)

```
p.innerHTML = userInput; // ⚠️ dangerous if userInput has <script>
p.textContent = userInput; // ✅ safe — always treated as plain text
```

⚠️ **Never use innerHTML with user-generated content.** Use `textContent` or `createTextNode` instead — these treat everything as plain text and prevent XSS (Cross-Site Scripting) attacks.

📦 `document.createDocumentFragment()`

Definition: A `DocumentFragment` is a lightweight, **in-memory container** that is not part of the live DOM tree. You build up multiple elements inside it, then insert the entire fragment in **one operation** — triggering only one reflow/repaint instead of many.

This is the performance trick React and virtual DOM libraries use internally on every render.

```
// ❌ Without fragment — 1000 separate DOM insertions = 1000 reflows
const ul = document.getElementById("list");
for (let i = 0; i < 1000; i++) {
  const li = document.createElement("li");
  li.textContent = `Item ${i}`;
  ul.appendChild(li); // each call triggers layout recalculation!
}
```

```
// ✅ With fragment — only 1 reflow at the end
const fragment = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
  const li = document.createElement("li");
  li.textContent = `Item ${i}`;
  fragment.appendChild(li); // no DOM, no reflow yet
}
ul.appendChild(fragment); // ONE insertion → one reflow. Fast!
```

💡 After insertion, the DocumentFragment becomes empty — its children **move** into the real DOM. The fragment itself disappears.

Category B — Inserting & Removing

[Old API] `appendChild()` — Add a child at the end

Definition: `parent.appendChild(node)` inserts a node as the **last child** of the parent. If the node already exists elsewhere in the DOM, it is **moved** (not copied) to the new location.

```
const ul = document.querySelector("ul");
const li = document.createElement("li");
li.textContent = "New Item";

ul.appendChild(li);
// <ul>...existing items...<li>New Item</li></ul>

// Moving an existing node
const existing = document.getElementById("someItem");
ul.appendChild(existing); // moves it to the end, doesn't duplicate
```

[Old API] `insertBefore()` — Insert before a specific child

Definition: `parent.insertBefore(newNode, referenceNode)` inserts a new node **before** the reference node. The reference node must be a direct child of the parent. If `referenceNode` is null, it behaves like `appendChild` (inserts at the end).

```
const list = document.querySelector("ul");
const newLi = document.createElement("li");
newLi.textContent = "First!";

const firstExisting = list.firstElementChild; // first <li>
list.insertBefore(newLi, firstExisting);
// <ul><li>First!</li> ...rest...</ul>

// Insert at end (null reference)
list.insertBefore(newLi, null); // same as appendChild
```

[Old API] removeChild() — Remove a child node

Definition: `parent.removeChild(child)` removes a **direct child** from the parent. You must have a reference to the parent. The removed node is returned — you can save and re-insert it later.

```
const list = document.querySelector("ul");
const item = document.querySelector("li.unwanted");

// Must call on the parent!
list.removeChild(item); // removes it, returns the removed node

// Common pattern: self-removal via parentNode
item.parentNode.removeChild(item); // works without explicitly knowing the
parent
```

[Modern API] append() — Insert multiple nodes + strings

Definition: `parent.append(...nodes)` is the modern upgrade to `appendChild`. It accepts **multiple arguments** and can mix DOM nodes and plain strings in a single call. Returns undefined (unlike `appendChild` which returns the inserted node).

```
const div = document.querySelector("div");
const span = document.createElement("span");
span.textContent = "World";

// Accepts nodes AND raw strings in one call
div.append("Hello ", span, "!");
// <div>Hello <span>World</span>!</div>

// appendChild would need 3 separate calls for the same result:
// div.appendChild(document.createTextNode("Hello "));
// div.appendChild(span);
// div.appendChild(document.createTextNode("!"));
```

[Modern API] prepend() / before() / after() / replaceWith()

Definitions:

parent.prepend(node) → inserts **inside** the parent as the **first child**
element.before(node) → inserts **outside**, as a sibling **before** the element
element.after(node) → inserts **outside**, as a sibling **after** the element
element.replaceWith(node) → **swaps** the element out entirely with the new node

```
const list = document.querySelector("ul");  
const item = document.querySelector("li"); // first li  
const newEl = document.createElement("li");  
newEl.textContent = "Prepended";
```

```
// prepend — inserts as FIRST child inside the parent  
list.prepend(newEl);  
// <ul><li>Prepended</li>...rest...</ul>
```

```
// before — inserts BEFORE the element (as a sibling, outside)  
const header = document.createElement("h3");  
list.before(header);  
// <h3></h3> <ul>...</ul>
```

```
// after — inserts AFTER the element (as a sibling, outside)  
const footer = document.createElement("p");  
list.after(footer);  
// <ul>...</ul> <p></p>
```

```
// replaceWith — swap the element for new content  
const ol = document.createElement("ol");  
list.replaceWith(ol);  
// <ul> is gone, <ol> takes its exact place
```

💡 before() and after() are called **on the element itself**, not on the parent. This is the key difference from insertBefore().

[Modern API] element.remove() — Self-removal

Definition: element.remove() lets an element **remove itself** from the DOM with no need to reference its parent. This is the cleanest, most readable way to delete a node.

```
const notification = document.querySelector(".toast");

// Old way (verbose — needs parent reference)
notification.parentNode.removeChild(notification);

// Modern way — clean self-removal 
notification.remove(); // gone. No parent reference needed.

// Real-world usage: auto-dismiss a toast after 3 seconds
setTimeout(() => {
  notification.remove();
}, 3000);
```

Quick Reference — Old API vs Modern API

Old API	Modern Equivalent	Key Difference
appendChild(node)	append(node)	append accepts multiple args + strings
insertBefore(n, ref)	ref.before(n)	before/after called on the reference element
insertBefore(n, null)	parent.prepend(n)	prepend for first-child insertion
parent.replaceChild(n, old)	old.replaceWith(n)	replaceWith called on the element itself
parent.removeChild(el)	el.remove()	No parent reference needed

 **Know both APIs cold.** Old API is everywhere in legacy codebases. Modern API is what you write in new code. React and frameworks use the same concepts under the hood on every render cycle.